



Ftrace

Part—2

In the previous article in this series, we had a working set-up of Ftrace, and explored options to enable and disable it. In this article, we will explore a few more capabilities of Ftrace.

Let's begin with tracer options. The output of the tracing can be controlled by a file called *trace_options*.

Various fields can be enabled and disabled by updating options in the file */sys/kernel/debug/tracing/trace_options*. A sample of *trace_options* can be viewed in Figure 1. To disable a tracing option, a 'no' keyword needs to be added to the start of the line. For example, `echo notrace_printk > trace_options`.

(Remember not to have a space between no and the option.) To enable a trace again, you could use, for instance, `echo trace_printk > trace_options`.

Ftrace for a specific process

Ftrace allows you to perform tracing even for a specific process. In the */sys/kernel/debug/tracing* directory, the file *set_ftrace_pid* needs to be updated with the PID of the process you want to be traced. The *traceprocess.sh* sample script below shows how to capture the PID on-the-go, and enable tracing. You can refine it with your own innovations. Run it with the command whose process you want to trace as the argument, as shown in Figure 2, where we traced the 'ls' command.

```
#!/bin/bash

DPATH="/sys/kernel/debug/tracing"
PID=$$

## Quick basic checks
[ `id -u` -ne 0 ] && { echo "needs to be root"
; exit 1; } # check for root permissions
[ -z $1 ] && { echo "needs process name as argument" ; exit 1; } # check for args to this function

mount | grep -i debugfs &> /dev/null
[ $? -ne 0 ] && { echo "debugfs not mounted, mount it first"; exit 1; } #checks for debugfs mount

# flush existing trace data
echo nop > $DPATH/current_tracer

# set function tracer
echo function > $DPATH/current_tracer

# enable the current tracer
echo 1 > $DPATH/tracing_enabled

# write current process id to set_ftrace_pid file
echo $PID > $DPATH/set_ftrace_pid

# start the tracing
echo 1 > $DPATH/tracing_on
```